



Kifaru Swypt

Virtual POS System

Comprehensive System Documentation
User Journey & Developer Architecture Guide

Version	Date	Author
1.0	March 2026	Chao

Empowering African merchants with stablecoin-powered commerce



Table of Contents

PART I: USER JOURNEY

1. Landing Page & Storefront
2. Merchant Authentication
3. Merchant Dashboard Overview
4. Supplier Management
5. Category Management
6. Inventory Management
7. Point of Sale (POS)
8. Payment & Checkout Flow
9. Blockchain Transaction Verification

PART II: DEVELOPER ARCHITECTURE

10. System Architecture Overview
11. Technology Stack
12. Database Schema
13. API Endpoints Reference
14. Authentication & Security
15. Payment Integration (Fonbnk)
16. Frontend Architecture
17. Deployment & Infrastructure

PART I

User Journey & Walkthrough

This section documents the complete user journey through the Kifaru Swypt Virtual POS system, from the initial landing page through merchant onboarding, inventory setup, and completing a sale with stablecoin payment. Each step is documented with a corresponding screenshot reference captured during a live walkthrough of the application.

1. Landing Page & Storefront

Step 1: Landing Page - Hero Section

Screenshot: Landing Page

The Kifaru Swypt homepage showing the hero section with tagline, trust indicators, and merchant CTA

The Kifaru Swypt landing page serves as the public-facing storefront and merchant onboarding entry point. The hero section prominently displays the platform's value proposition: **"The Smart Way to Manage & Get Paid"** with a subtitle emphasizing virtual POS capabilities and stablecoin payment integration.

The page features a fixed header with the Kifaru Swypt branding, navigation links (Features, Solutions, Pricing), a shopping cart icon, and authentication buttons (Log In / Get Started). Below the hero, four trust indicator cards highlight key platform features:

- **Instant Settlements** - Receive stablecoins directly in your wallet
- **Secure Transactions** - Blockchain-verified payment security
- **Low Fees** - Fraction of the cost of traditional card networks
- **Inventory Sync** - Real-time tracking across all locations

A prominent call-to-action section at the bottom invites merchants to sign up or log in, with the message **"Ready to transform your business?"** The footer provides platform feature links, merchant support information, and Swypt payment branding.

2. Merchant Authentication

Step 2: Merchant Login

Screenshot: Login Page

The merchant login form with 'Welcome Back' header, email field, password field, and Log In button

The merchant login page presents a clean, centered authentication form with the Kifaru logo and **"Welcome Back"** heading, subtitled **"Virtual POS Merchant Login"**. The form includes:

- **Merchant Email** - Email input field with a mail icon prefix
- **Password** - Masked password field with a lock icon and visibility toggle (eye icon)
- **Log In Button** - Full-width green gradient button with sparkle icon
- **Sign Up Link** - "Don't have a merchant account? Sign up instead" for new merchants
- **Home Link** - "Back to Kifaru Swypt Home" link at the bottom

Authentication Flow: Upon successful login, the system generates a JWT token (valid for 7 days) which is stored in the browser's localStorage. The token is automatically included in all subsequent API requests via an Axios interceptor, enabling seamless authenticated access to the merchant dashboard.

Step 2b: Login Form - Complete View

Screenshot: Login Form (Full View)

Complete login form showing the Sign Up link and Back to Home navigation

3. Merchant Dashboard Overview

Step 3: Dashboard - Main View

Screenshot: Merchant Dashboard

Dashboard with sidebar navigation, quick action cards, wallet address, and recent products table

After successful authentication, merchants are redirected to their dashboard, which serves as the central command hub for all business operations. The dashboard features a persistent left sidebar navigation and a content area with quick-access cards and data summaries.

Sidebar Navigation

- **POS / New Sale** - Point-of-sale interface for processing sales
- **Dashboard** - Overview and quick actions (currently active)
- **Inventory** - Stock management, product analytics, and alerts
- **Categories** - Product category hierarchy management
- **Reports** - Sales and inventory reporting
- **Suppliers** - Supplier relationship management
- **Transactions Log** - Complete transaction history
- **My Wallet** - Blockchain wallet address management

Quick Action Cards

The dashboard displays four quick-action cards: **View Transactions**, **View Orders**, **View Wallet Address**, and a prominently styled green **Add Product** button. Below the cards, the merchant's wallet address is displayed with a copy-to-clipboard function.

Recent Products Table

Screenshot: Dashboard - Products List (Scrolled)

Paginated product table showing Image, Name, Price, Category, Supplier, Quantity, and Edit/Delete actions

The recent products table provides a paginated overview of all inventory items with columns for Image, Name, Price, Category, Supplier, Quantity, and Actions (Edit / Delete). Products are

displayed with their images, and pagination controls allow navigation through larger inventories.

4. Supplier Management

Onboarding Note: For a new merchant, supplier setup is the recommended first step. Suppliers must be created before products can be linked to them during inventory setup.

Step 4: Suppliers List

Screenshot: Suppliers Page

Suppliers table with Name, Email, Phone, Status (Active), and action icons (deactivate, edit, delete)

The Suppliers page displays all registered suppliers in a clean data table with columns for **Name**, **Email**, **Phone**, **Status**, and **Actions**. Each supplier has a status badge (Active/Inactive) and action buttons for deactivating, editing, or deleting. An "All Status" filter dropdown at the top allows filtering suppliers by their active status.

Step 4b: Add New Supplier

Screenshot: Add Supplier Modal

Modal form with Name, Contact Email*, Phone, Address, Status dropdown, and Save/Cancel buttons*

Clicking the green "+ Add Supplier" button opens a modal form with the following fields:

Field	Type	Required	Description
Name	Text input	Yes	Supplier's business name
Contact Email	Email input	Yes	Primary contact email address
Phone	Text input	No	Phone number
Address	Textarea	No	Physical address
Status	Dropdown	No	Active (default) or Inactive

5. Category Management

Step 5: Categories List

Screenshot: Categories Page

Hierarchical categories table with Name, Description, Parent Category, Products count, and Edit/Delete actions

The Categories page supports a **hierarchical category structure**, allowing merchants to organize products into parent categories and subcategories. The table displays Name, Description, Parent Category, product count, and action buttons. Examples include "Accessories" under "Electronics", "Baby Care" and "Baby Food" under "Baby & Kids", and "Bakery" under "Fresh Food".

Step 5b: Add New Category

Screenshot: Add Category Modal

Modal form with Name, Description, Parent Category (optional dropdown), and Save/Cancel buttons*

The Add Category form supports creating both top-level categories and subcategories through an optional **Parent Category** dropdown. Required fields are marked with a red asterisk. The Description field allows merchants to add contextual information about the category.

6. Inventory Management

Step 6: Inventory Overview Dashboard

Screenshot: Inventory Overview

Inventory dashboard with summary cards (Total Products, Stock Value, Low Stock, Out of Stock), action buttons, Top 10 Products chart, and Low Stock Alerts panel

The Inventory page provides a comprehensive overview of the merchant's stock with real-time analytics:

Summary Cards

- **Total Products** (purple gradient) - Total number of products in inventory (e.g., 7)
- **Total Stock Value** (green gradient) - Aggregate monetary value of all stock (e.g., KSH 124,723.00)
- **Low Stock Items** (yellow/warning) - Products below reorder threshold (e.g., 2)
- **Out of Stock** (red/gradient) - Products with zero quantity (e.g., 0)

Action Buttons

Four action buttons provide quick access: **+ Add New Product**, **Adjust Stock** (wrench icon), **View Alerts** (with count badge), and **Export Report**.

Analytics Panels

The **Top 10 Products by Stock** horizontal bar chart visualizes inventory levels across products. The **Low Stock Alerts** panel displays real-time notifications when products fall below defined thresholds.

Step 6b: Add New Product Form

Screenshot: Add Product Form

Product creation form with Basic Information (Name, Description, Category, SKU) and Product Image upload area

The Add Product form is a full-page layout divided into sections:

Section	Field	Required	Description
Basic Info	Product Name	Yes	Product display name
Basic Info	Description	No	Product features and details
Basic Info	Category	Yes	Dropdown linked to Categories
Basic Info	SKU	No	Stock Keeping Unit identifier
Image	Product Image	Yes	Drag-and-drop upload (single image)
Pricing	Price	Yes	Product price in KES
Pricing	Quantity	Yes	Initial stock quantity
Pricing	Supplier	Yes	Linked supplier from Suppliers list

7. Point of Sale (POS)

Step 7: POS Interface - Empty Cart

Screenshot: POS - Empty Cart

Split-panel POS with product grid (left) showing items with images, prices, stock levels; and Current Sale panel (right) showing 'No items in cart', Total: KES 0

The POS (Point of Sale) interface is the core transactional screen where merchants process customer purchases. It uses a **two-panel layout**:

- **Left Panel - Product Grid:** Displays all available products as visual cards with product images, names, prices (in green), and current stock levels. A search bar at the top enables quick product filtering. Clicking a product card adds it to the current sale.
- **Right Panel - Current Sale:** Shows the active shopping cart with line items, quantity controls (+/- buttons), delete buttons, running total in KES, and the green "Charge Customer" button to proceed to checkout.

Step 7b: POS Interface - Items in Cart

Screenshot: POS - Items Added to Cart

POS with Canned Pineapple (qty 8, @1) and Pineapple (qty 5, @250) in cart. Total: KES 258

In this example, the merchant has added two products to the current sale: **Canned Pineapple** (quantity: 8, unit price: KES 1) and **Pineapple** (quantity: 5, unit price: KES 50). The cart displays each item with its thumbnail image, name, unit price, quantity with +/- adjustment controls, and a delete button. The total automatically updates to **KES 258**.

8. Payment & Checkout Flow

Sandbox Mode Notice: The payment flow demonstrated here uses the Fonbnk sandbox environment. All transactions use testnet funds on the Celo Sepolia Testnet. No real money is involved. In production, the same flow would process actual M-Pesa payments converted to USDT stablecoins.

Step 8a: Email Verification

Screenshot: Checkout - Email Verification

Kifaru POS modal with green header (KES 258), email input, Cloudflare verification, and 'Next: Email Verification' button

After clicking "Charge Customer", the **Kifaru POS payment modal** opens with a green gradient header displaying "Secure M-Pesa Payment" and the total amount (KES 258). The first step requires the customer to enter their email address. A **Cloudflare verification** check runs to confirm the user is human. The "Next: Email Verification" button proceeds to order review.

Step 8b: Order Review & Crypto Conversion

Screenshot: Checkout - Order Review

Order details showing Local currency: 258 KES, Crypto amount: 1.944982 USDT, Wallet: 0xb0e...ff43, and USDT/Celo confirmation checkbox

The customer reviews their order showing the real-time currency conversion:

Field	Value	Description
Local Currency Amount	258 KES	Kenyan Shilling amount to pay
Crypto Amount	1.944982 USDT	Equivalent stablecoin value
Network	Celo	Blockchain network for settlement
Wallet	0xb0e...ff43	Merchant's receiving wallet address

A mandatory confirmation checkbox requires the customer to acknowledge they are purchasing **USDT** cryptocurrency on the **Celo** network from Fonbnk and confirm control of the receiving

wallet address.

Step 8c: M-Pesa OTP Verification

Screenshot: Checkout - OTP Verification

Amount to Send: KES 258, sandbox OTP instructions (123456), OTP input field, Verify button, and 5-minute cancellation notice

The payment proceeds to the M-Pesa transfer stage. The modal displays the **Amount to Send: KES 258** with a copy button. In the sandbox environment, the system instructs users to enter **123456** as the OTP code. The "Verify OTP code" button submits the verification. An important notice warns that **non-confirmed orders will be automatically canceled after 5 minutes**.

Step 8d: Order In Progress

Screenshot: Checkout - Order In Progress

Loading state with 'Waiting for the agent to confirm that they received funds', Order ID: 69ba6...2765, amount: 1.944981 USDT

After OTP verification, the system enters a processing state showing **"Order In Progress"** with the message "Waiting for the agent to confirm that they received funds." A loading spinner indicates active processing. The order details are displayed including the unique Order ID, expected USDT amount, and destination wallet. A "Report a problem" option is available for issue escalation.

Step 8e: Order Completed

Screenshot: Checkout - Order Completed

Success confirmation: '1.944981 USDT was successfully transferred to your wallet', with View on block explorer and Report a problem links

Upon successful payment, the modal displays **"Order Completed"** with confirmation that 1.944981 USDT was successfully transferred to the merchant's wallet. The customer can **"View on block explorer"** to verify the on-chain transaction, or "Report a problem" if needed. The Order ID and received amount are displayed for record-keeping.

9. Blockchain Transaction Verification

Step 9: Celo Block Explorer - Transaction Details

Screenshot: Celo Sepolia Testnet Block Explorer

CeloScan transaction details: Transfer 1.944981 USD, Status: Success, Block: 20540001, with full transaction hash and wallet address

Clicking "View on block explorer" opens the **Celo Sepolia Testnet** block explorer (CeloScan) showing the complete on-chain transaction record. This provides immutable proof of payment:

Field	Value
Transaction Action	Transfer 1.944981 USD to merchant wallet
Network	Celo Sepolia Testnet
Status	Success (green checkmark)
Block	20540001 (Confirmed by Sequencer)
Tabs Available	Overview, Logs (1), State

Blockchain Transparency: Every payment processed through Kifarú Swypt is recorded on the Celo blockchain, providing an immutable, transparent audit trail. Both merchants and customers can independently verify transactions using the block explorer. In production, transactions occur on the Celo mainnet.

PART II

Developer Architecture & Technical Reference

This section provides a technical deep-dive into the Kifaru Swypt architecture, covering the technology stack, database schema, API endpoints, authentication mechanisms, payment integration, and deployment configuration. This serves as a reference for developers maintaining or extending the system.

10. System Architecture Overview

Kifaru Swypt follows a **monorepo architecture** with clearly separated frontend and backend services that communicate via RESTful APIs. The system integrates with the Fonbnk payment gateway for stablecoin payments and uses PostgreSQL for persistent data storage.

Layer	Technology	Purpose
Frontend	Next.js 14 (App Router)	Server-rendered React UI with TailwindCSS
Backend	Express 5.1 (TypeScript)	RESTful API with JWT authentication
Database	PostgreSQL	Relational data store with JSONB support
Payments	Fonbnk API + Celo Blockchain	M-Pesa to USDT stablecoin conversion
Image Storage	Cloudinary	Product image hosting and CDN delivery
Deployment	Vercel (FE) + Railway (BE)	Managed cloud infrastructure

Project Structure

The monorepo is organized under **kifaruSwypt/** with two main directories: **kifaruBackend/** (Express API) and **kifaruFrontend/** (Next.js app). Database migrations are stored in **migrations/**. Docker and deployment configs sit at the repository root.

11. Technology Stack

Backend Dependencies

Package	Version	Purpose
express	5.1.0	Web framework

pg (node-postgres)	Latest	PostgreSQL client
jsonwebtoken	Latest	JWT token generation/verification
bcrypt	Latest	Password hashing (10 salt rounds)
cors	Latest	Cross-origin request handling
joi	Latest	Request validation schemas
uuid	Latest	UUID generation for IDs
dotenv	Latest	Environment variable management
typescript	5.x	Type-safe development

Frontend Dependencies

Package	Purpose
next (14.x)	React framework with App Router
tailwindcss	Utility-first CSS framework
antd (Ant Design)	UI component library (tables, modals, forms)
axios	HTTP client with interceptors
lucide-react	Icon library
cloudinary	Image upload integration

12. Database Schema

The PostgreSQL database uses a relational schema with proper foreign key constraints and JSONB fields for flexible metadata storage. All tables use UUID primary keys for security and scalability.

Table	Key Fields	Purpose
merchants	merchant_id (UUID), username, email, password_hash, store_name, wallet_address	Merchant accounts with blockchain wallet
Products	id, merchant_id, name, price, quantity, imageurl, category_id, supplier_id	Product catalog with inventory tracking
Categories	id, merchant_id, name, parent_id	Hierarchical product category tree
Suppliers	id, merchant_id, name, contact_email, phone, is_active	Supplier relationship management
StockMovements	id, product_id, movement_type, quantity_change, created_at	Inventory audit trail (IN/OUT/SALE/RETURN)
InventoryAlerts	id, product_id, merchant_id, alert_type, message, is_read	Low stock and status notifications
Transactions	id, merchant_id, total_amount, currency, status, payment_metadata (JSONB)	Order records with payment details

Key Design Decisions

- **UUID Primary Keys:** All tables use UUID identifiers for security (non-guessable) and distributed compatibility
- **JSONB Fields:** payment_metadata and customer_details use JSONB for schema flexibility without migrations
- **Hierarchical Categories:** Self-referencing parent_id enables nested category trees
- **Stock Movements:** Every inventory change is logged as an immutable audit record with movement types (IN, OUT, ADJUSTMENT, SALE, RETURN)
- **ACID Transactions:** Checkout operations use PostgreSQL transactions for atomicity - stock adjustments and order creation happen together or not at all

13. API Endpoints Reference

Authentication & User Management

Method	Endpoint	Auth	Description
POST	/api/signup	Public	Register new merchant
POST	/api/auth/login	Public	Login, returns JWT token
GET	/api/getWallet/:id	JWT	Fetch merchant wallet address
POST	/api/saveWallet	JWT	Store wallet address
POST	/api/AddProduct	JWT	Create a new product
GET	/api/getProducts	Public	List all products (storefront)
GET	/api/getMerchantProducts/:id	JWT	Get merchant's products
PUT	/api/updateProduct/:id	JWT	Update product details
DELETE	/api/deleteProduct/:id	JWT	Delete a product

Inventory Management

Method	Endpoint	Auth	Description
POST	/api/inventory/categories	JWT	Create category
GET	/api/inventory/categories	JWT	List all categories
PUT	/api/inventory/categories/:id	JWT	Update category
DELETE	/api/inventory/categories/:id	JWT	Delete category
POST	/api/inventory/suppliers	JWT	Create supplier
GET	/api/inventory/suppliers	JWT	List all suppliers
PUT	/api/inventory/suppliers/:id	JWT	Update supplier
DELETE	/api/inventory/suppliers/:id	JWT	Delete supplier

POST	/api/inventory/adjust	JWT	Adjust stock levels
GET	/api/inventory/report	JWT	Generate inventory reports
POST	/api/inventory/checkout	Public	Process sale & adjust stock

Payment Integration

Method	Endpoint	Auth	Description
POST	/api/swypt/widget-url	JWT	Generate signed Fonbnk widget URL
POST	/api/swypt/webhook	Signature	Fonbnk payment status webhook

14. Authentication & Security

JWT Implementation

Parameter	Value
Algorithm	HS256 (HMAC-SHA256)
Expiration	7 days
Payload	{ merchant_id, email }
Secret	JWT_SECRET environment variable
Storage	localStorage (key: merchantToken)
Transport	Authorization: Bearer header

Authentication Flow

1. User submits email and password via the login form
2. Backend queries the merchants table for matching email
3. `bcrypt.compare()` validates the submitted password against the stored hash
4. On success, a JWT token is generated with `merchant_id` and email in the payload
5. Token is returned to the frontend and stored in `localStorage`
6. Axios interceptor automatically attaches the token as a Bearer header on all API requests
7. Protected routes use the `VerifyToken` middleware to decode and validate the JWT
8. On token expiry or invalidity, the user is redirected to the login page

Security Measures

- **Password Hashing:** `bcrypt` with 10 salt rounds - passwords never stored in plaintext
- **CORS Configuration:** Restricted to allowed origins via `ALLOWED_ORIGINS` environment variable
- **Webhook Signature Verification:** `SHA256(body + SHA256(secret))` validates Fonbnk webhooks
- **Protected Routes:** JWT middleware on all sensitive endpoints (inventory, wallet, products)
- **Public Endpoints:** Only signup, login, product listing, and checkout are public

15. Payment Integration (Fonbnk)

Kifar Swypt integrates with **Fonbnk** to enable M-Pesa to USDT (stablecoin) payment conversion. The integration uses a widget-based approach where Fonbnk handles the payment UI within an iframe, while Kifar manages the order lifecycle and inventory adjustments.

Payment Flow (Technical)

- 1. Widget URL Generation** - Frontend calls POST `/api/swypt/widget-url` with amount and merchant_id. Backend fetches merchant wallet from DB, signs a JWT with a unique uid and wallet address, and returns a signed URL pointing to `https://sandbox-pay.fonbnk.com/auto-order`
- 2. Widget Configuration** - The URL is configured with: network=CELO, asset=USDT, currency=KES, paymentChannel=mobile_money, flow=onramp, freezeWallet=true, freezeAmount=true
- 3. Iframe Integration** - FonbnkCheckoutModal.tsx opens the widget URL in an iframe and listens for postMessage events including 'order-created' and 'close-iframe'
- 4. M-Pesa Payment** - Customer completes M-Pesa STK push on their phone, submits OTP for verification
- 5. Blockchain Settlement** - Fonbnk converts KES to USDT on the Celo blockchain and transfers to merchant's wallet address
- 6. Webhook Notification** - Fonbnk sends POST `/api/swypt/webhook` with payment status. Backend verifies x-signature header using SHA256, updates transaction status to COMPLETED
- 7. Inventory Sync** - On success callback, frontend calls POST `/api/inventory/checkout` to deduct stock and create a transaction record within a PostgreSQL transaction for ACID compliance

Fonbnk Configuration

Parameter	Value	Description
FONBNK_BASE_URL	sandbox-pay.fonbnk.com	Payment gateway URL (sandbox)
FONBNK_SOURCE	Merchant identifier	Source identifier for Fonbnk
FONBNK_URL_SIGNATURE_SECRET	Secret key	Signs widget URLs (HS256)
FONBNK_API_SIGNATURE_SECRET	Secret key	Verifies webhook signatures
Network	CELO	Blockchain for USDT settlement

Asset	USDT	Stablecoin for payments
Currency	KES	Local fiat currency (M-Pesa)

16. Frontend Architecture

The frontend is built with **Next.js 14** using the App Router pattern. It uses a combination of TailwindCSS for styling and Ant Design for data-heavy components (tables, forms, modals). State management relies on React hooks with localStorage for authentication persistence.

Route Structure

Route	Access	Description
/	Public	Landing page / storefront with product catalog
/merchants/login	Public	Merchant login form
/merchants/signUp	Public	Merchant registration form
/merchants/dashboard	Protected	Main dashboard with product overview
/merchants/dashboard/inventory	Protected	Stock management and analytics
/merchants/dashboard/categories	Protected	Category CRUD operations
/merchants/dashboard/suppliers	Protected	Supplier management
/merchants/dashboard/pos	Protected	Point of sale interface
/merchants/dashboard/transactions	Protected	Transaction history
/merchants/dashboard/reports	Protected	Sales and inventory reports

Key Components

- **FonbnkCheckoutModal** - Iframe wrapper for Fonbnk payment widget with postMessage event handling
- **CloudinaryUpload** - Image upload component integrated with Cloudinary CDN
- **WalletSetupModal** - Blockchain wallet address configuration for merchants
- **api.ts (Axios instance)** - Centralized HTTP client with JWT token interceptor for automatic authentication

17. Deployment & Infrastructure

Production Deployment

Service	Platform	Configuration
Frontend	Vercel	Auto-deploy from Git, pnpm build, Frozen lockfile for consistency
Backend	Railway	Node.js runtime, npm build, Auto-scaling with health checks
Database	PostgreSQL (managed)	Connection pool via pg driver, environment-configured

Docker Support

The project includes Docker configuration for local development and alternative deployment. The **docker-compose.yaml** defines two services: frontend (port 3500 mapped to 3000) and backend (port 4000). Both use a **Node 22 slim** base image.

Environment Variables

Variable	Service	Purpose
PORT	Backend	Server listening port
PG_DATABASE, PG_USER, PG_HOST, PG_PASSWORD, PG_PORT	Backend	PostgreSQL connection details
JWT_SECRET	Backend	JWT token signing secret
ALLOWED_ORIGINS	Backend	CORS allowed origins
FONBNK_BASE_URL	Backend	Fonbnk payment gateway URL
FONBNK_SOURCE	Backend	Fonbnk merchant source ID
FONBNK_URL_SIGNATURE_SECRET	Backend	Widget URL signing secret
FONBNK_API_SIGNATURE_SECRET	Backend	Webhook signature verification
NEXT_PUBLIC_API_BASE_URL	Frontend	Backend API URL for client-side calls

End of Documentation

Kifaru Swypt Virtual POS System v1.0

Empowering merchants with blockchain-powered commerce

This document was generated as part of the Kifaru Swypt Virtual POS system documentation. For questions or updates, contact the development team.

Document generated: March 2026